

Q1. What is SQL and why is it important in data science?

Answer: SQL (Structured Query Language) is used to store, retrieve, and manipulate data in relational databases. In data science, it is the primary tool to extract and filter large datasets before analysis or modeling.

Q2. What is the difference between a database and a table in SQL?

Answer: A database is a collection of organized data that holds multiple tables. A table is a single structured object inside a database that stores data in rows and columns.

Q3. What is a primary key and why must it be unique in a table?

Answer: A primary key is a column that uniquely identifies each row in a table. It must be unique so that no two rows have the same identity, which prevents duplicate or ambiguous records.

Q4. What is a foreign key and how does it create a relationship between two tables?

Answer: A foreign key is a column in one table that refers to the primary key of another table. It creates a link between the two tables, enforcing referential integrity in the database.

Q5. What is the difference between WHERE and HAVING clause in SQL?

Answer: WHERE filters rows before grouping and cannot use aggregate functions. HAVING filters groups after GROUP BY and is used with aggregate functions like COUNT or SUM.

Q6. What is a JOIN and what are the different types of joins available in SQL?

Answer: A JOIN combines rows from two or more tables based on a related column. The main types are INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN, and CROSS JOIN.

Q7. What is the difference between INNER JOIN and LEFT JOIN in SQL?

Answer: INNER JOIN returns only rows that have matching values in both tables. LEFT JOIN returns all rows from the left table and matching rows from the right table, with NULL for non-matching right rows.

Q8. What is a GROUP BY clause and how does it work with aggregate functions?

Answer: GROUP BY groups rows that have the same values in specified columns into summary rows. Aggregate functions like COUNT, SUM, AVG are then applied to each group separately.

Q9. What is the difference between COUNT(*) and COUNT(column_name) in SQL?

Answer: COUNT(*) counts all rows including those with NULL values. COUNT(column_name) counts only the rows where that specific column is not NULL.

Q10. What is a subquery and when would you use it instead of a join?

Answer: A subquery is a query nested inside another query. It is used when the result of one query is needed as a condition or input for another, especially when a join would be too complex or unreadable.

Q11. What is the difference between UNION and UNION ALL in SQL?

Answer: UNION combines results of two queries and removes duplicate rows. UNION ALL combines results and keeps all duplicates. UNION ALL is faster because it skips the deduplication step.

Q12. What is a NULL value and how is it different from zero or an empty string?

Answer: NULL means the value is unknown or missing — it has no value at all. Zero is an actual numeric value and an empty string is a string with no characters. NULL cannot be compared using = or !=.

Q13. What is the difference between IS NULL and = NULL in a SQL query?

Answer: IS NULL is the correct way to check for NULL values in SQL. = NULL always returns false because NULL is not equal to anything, including itself. Always use IS NULL or IS NOT NULL.

Q14. What are aggregate functions and name five commonly used ones in SQL?

Answer: Aggregate functions perform a calculation on a set of rows and return a single value. The five most common are COUNT(), SUM(), AVG(), MIN(), and MAX().

Q15. What is the difference between DELETE, DROP, and TRUNCATE in SQL?

Answer: DELETE removes specific rows based on a condition and can be rolled back. TRUNCATE removes all rows from a table quickly and cannot be rolled back. DROP deletes the entire table structure along with its data.

Q16. What is a view in SQL and how is it different from a table?

Answer: A view is a saved SQL query that acts like a virtual table. Unlike a real table, it does not store data physically — it pulls data from underlying tables every time it is queried.

Q17. What is normalization and why is it important in database design?

Answer: Normalization is the process of organizing a database to reduce data redundancy and improve data integrity. It splits large tables into smaller related ones and ensures each piece of data is stored only once.

Q18. What is the difference between 1NF, 2NF, and 3NF in normalization?

Answer: 1NF ensures each column has atomic values and no repeating groups. 2NF removes partial dependencies on a composite primary key. 3NF removes transitive dependencies where non-key columns depend on other non-key columns.

Q19. What is an index in SQL and how does it improve query performance?

Answer: An index is a data structure that speeds up data retrieval on a column. It works like a book index — instead of scanning every row, SQL uses the index to jump directly to matching rows.

Q20. What is the difference between a clustered and non-clustered index?

Answer: A clustered index sorts and stores the actual data rows in the table based on the index key — only one is allowed per table. A non-clustered index creates a separate structure pointing to the data rows, and multiple can exist.

Q21. What is the ORDER BY clause and how do ASC and DESC work with it?

Answer: ORDER BY sorts the result set based on one or more columns. ASC sorts in ascending order (smallest to largest) which is the default. DESC sorts in descending order (largest to smallest).

Q22. What is the difference between RANK(), DENSE_RANK(), and ROW_NUMBER() window functions?

Answer: ROW_NUMBER() assigns a unique number to every row. RANK() gives the same rank to ties but skips the next number. DENSE_RANK() gives the same rank to ties but does not skip any numbers.

Q23. What is a window function and how is it different from a regular aggregate function?

Answer: A window function performs a calculation across a set of rows related to the current row without collapsing them into a single output row. A regular aggregate function like SUM collapses all rows into one result.

Q24. What is the PARTITION BY clause and how does it work inside a window function?

Answer: PARTITION BY divides the result set into groups (partitions) and the window function is applied independently to each partition. It is like GROUP BY but does not reduce the number of rows in the output.

Q25. What is a CTE (Common Table Expression) and how is it different from a subquery?

Answer: A CTE is a temporary named result set defined with the WITH keyword before the main query. Unlike a subquery, a CTE is more readable, can be referenced multiple times in the same query, and supports recursion.

Q26. What is the difference between CHAR, VARCHAR, and TEXT data types in SQL?

Answer: CHAR is a fixed-length string that always uses the defined space. VARCHAR is a variable-length string that uses only as much space as needed. TEXT is used for very large strings with no defined length limit.

Q27. What is a stored procedure and how is it different from a function in SQL?

Answer: A stored procedure is a precompiled block of SQL code that can perform actions like INSERT or UPDATE and may or may not return a value. A function must return a value and can be used inside a SELECT statement.

Q28. What is a trigger in SQL and when would it be used?

Answer: A trigger is a block of SQL code that automatically executes in response to an INSERT, UPDATE, or DELETE event on a table. It is used to enforce rules, log changes, or maintain data consistency automatically.

Q29. What is the difference between CROSS JOIN and FULL OUTER JOIN?

Answer: CROSS JOIN returns every combination of rows from both tables (Cartesian product) with no condition. FULL OUTER JOIN returns all rows from both tables, filling NULL where there is no match on either side.

Q30. What is the CASE statement in SQL and how does it work like an if-else?

Answer: CASE evaluates a list of conditions and returns a value for the first true condition, similar to if-elif-else in programming. It is used inside SELECT to create conditional columns in query results.

Q31. What is the difference between LIKE and = operator when filtering string data?

Answer: = checks for an exact match of the full string. LIKE is used for pattern matching with wildcards like % and _, allowing partial or flexible string comparisons in a WHERE clause.

Q32. What are wildcards in SQL and what is the difference between % and _?

Answer: Wildcards are special characters used with LIKE for pattern matching. % matches zero or more characters of any kind. _ matches exactly one single character.

Q33. What is the DISTINCT keyword and how does it affect query results?

Answer: DISTINCT removes duplicate rows from the result set and returns only unique values. It is placed right after SELECT and applies to all columns listed in the query.

Q34. What is the difference between BETWEEN and using >= and <= together?

Answer: Both produce the same result — BETWEEN is just a cleaner, more readable shorthand. BETWEEN is inclusive on both ends, same as using >= lower_value AND <= upper_value.

Q35. What is a self join and when would a data scientist use it?

Answer: A self join is when a table is joined with itself using aliases. A data scientist uses it to compare rows within the same table, such as finding employees who report to the same manager or comparing time-based records.

Q36. What is the COALESCE() function and how is it used to handle NULL values?

Answer: COALESCE() returns the first non-NULL value from a list of arguments. It is commonly used to replace NULL with a default value, making data cleaner for analysis and reporting.

Q37. What is the difference between OLTP and OLAP databases in data science context?

Answer: OLTP (Online Transaction Processing) databases are optimized for fast insert, update, and delete operations used in daily business. OLAP (Online Analytical Processing) databases are optimized for complex read queries and aggregations used in data analysis.

Q38. What is a fact table and a dimension table in a data warehouse schema?

Answer: A fact table stores measurable, quantitative data like sales amounts or order counts. A dimension table stores descriptive attributes like product name, date, or customer details that give context to the facts.

Q39. What is the difference between a star schema and a snowflake schema?

Answer: In a star schema, dimension tables are directly connected to the fact table in a simple star shape. In a snowflake schema, dimension tables are normalized and split into sub-dimensions, making it more complex but reducing redundancy.

Q40. What is the EXPLAIN or EXPLAIN ANALYZE command and why is it useful?

Answer: EXPLAIN shows the execution plan SQL uses to run a query — including which indexes are used and the order of operations. EXPLAIN ANALYZE actually runs the query and shows real timing, helping identify and fix slow queries.

Q41. What is query optimization and what are common techniques used in SQL?

Answer: Query optimization is the process of rewriting queries to run faster and use fewer resources. Common techniques include using indexes, avoiding SELECT *, filtering early with WHERE, avoiding functions on indexed columns, and using CTEs instead of repeated subqueries.

Q42. What is the difference between correlated and non-correlated subqueries?

Answer: A non-correlated subquery runs independently once and its result is used by the outer query. A correlated subquery references a column from the outer query and re-executes for every row, making it slower.

Q43. What is the LAG() and LEAD() function and how are they used in time series analysis?

Answer: LAG() returns the value of a column from a previous row within the partition. LEAD() returns the value from a following row. Both are used in time series to compare current values with past or future values, like calculating day-over-day change.

Q44. What is the NTILE() function and how is it used to divide data into buckets?

Answer: NTILE(n) divides the rows in a partition into n equal-sized buckets and assigns each row a bucket number from 1 to n. Data scientists use it to create quartiles, deciles, or custom percentile groups.

Q45. What is the difference between INNER JOIN and CROSS JOIN in terms of output rows?

Answer: INNER JOIN returns only rows where the join condition is matched between tables. CROSS JOIN returns all possible combinations of rows from both tables with no condition, so output rows = rows in table A × rows in table B.

Q46. What is a materialized view and how is it different from a regular view?

Answer: A materialized view physically stores the result of the query on disk and refreshes it periodically. A regular view is virtual and always recomputes from base tables on each query. Materialized views are faster for heavy aggregation queries.

Q47. What is ROLLUP and CUBE in SQL and how do they extend GROUP BY?

Answer: ROLLUP generates subtotals and a grand total along a hierarchy of columns. CUBE generates subtotals for all possible combinations of columns. Both are extensions of GROUP BY used for multi-dimensional summary reports in data analysis.

Q48. What is the difference between horizontal and vertical scaling in SQL databases?

Answer: Vertical scaling means adding more power (CPU, RAM) to a single server. Horizontal scaling means adding more servers and distributing the data across them. SQL databases are traditionally easier to scale vertically; horizontal scaling requires sharding.

Q49. What is a transaction in SQL and what are the ACID properties?

Answer: A transaction is a sequence of SQL operations that are executed as a single unit. ACID stands for Atomicity (all or nothing), Consistency (data stays valid), Isolation (transactions don't interfere), and Durability (changes persist after commit).

Q50. What is the difference between NoSQL and SQL databases and when would a data scientist choose one over the other?

Answer: SQL databases store structured data in tables with a fixed schema and are ideal for relational data and complex queries. NoSQL databases store unstructured or semi-structured data like JSON and scale better horizontally. A data scientist chooses SQL for structured analytics and NoSQL for large-scale unstructured data like logs or documents.